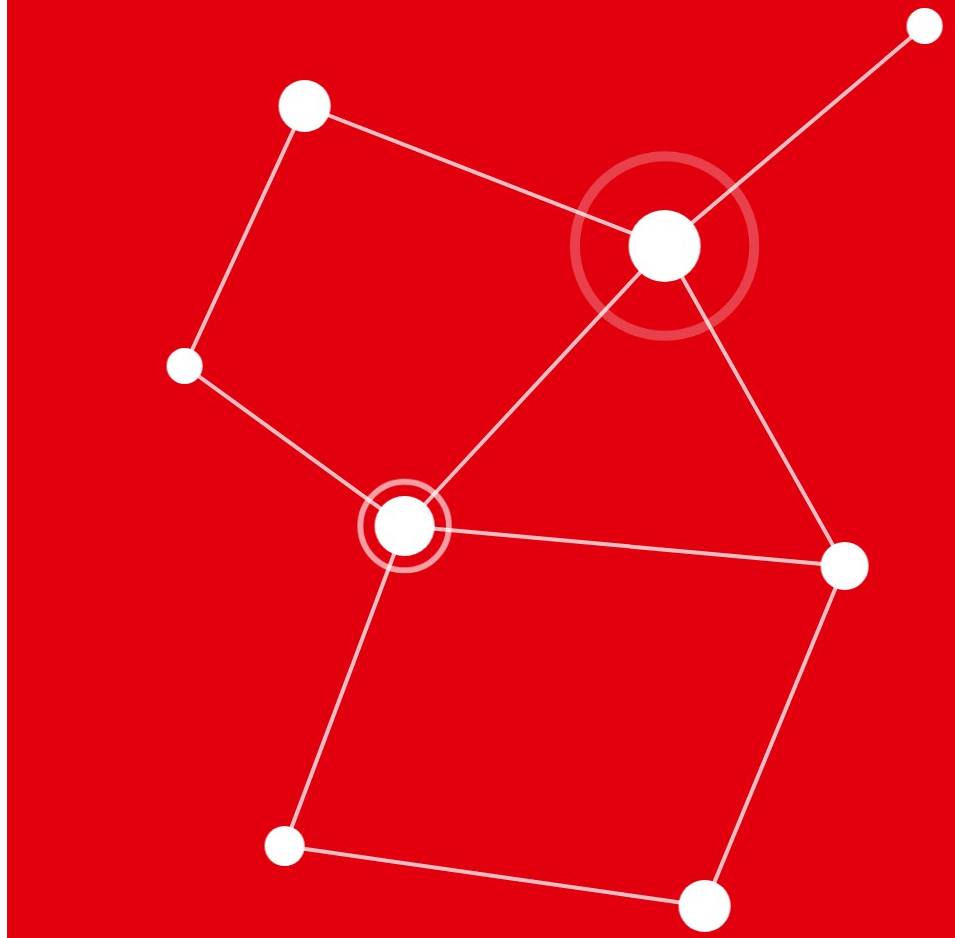
ENGINEERING KNOWLEDGE · TECH TALK

From document decay to living knowledge

The LLM Wiki pattern, the Open Knowledge Format (OKF), and an agent-maintained knowledge base you can adopt today.

LLM Wiki × OKF × knowledgeAgent · November 2026



WHERE THIS STARTS

April 2026: a short “idea file” on GitHub Gist

AK **karpathy** / llm-wiki.md Apr 4, 2026

LLM Wiki

A pattern for building personal knowledge bases with LLMs — written to be pasted into an agent, which then works out the details with you.

CORE IDEA

Don’t retrieve from raw documents at query time. Let the LLM **incrementally build and maintain** a persistent, interlinked markdown wiki that sits between you and your sources.

THREE LAYERS · THREE OPERATIONS

Raw sources · Wiki · Schema — Ingest · Query · Lint

gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

> The wiki is **“a persistent, compounding artifact.”**

- You **curate sources and ask questions**; the LLM does the writing and the upkeep.
- Humans abandon wikis because **maintenance grows faster than value**. LLMs do the bookkeeping at near-zero cost.
- Loosely echoes Vannevar Bush’s **Memex** (1945) — an associative, personal store of knowledge.

Why did it resonate so widely? Because every engineering team already lives with the problem it describes. **Let’s start there.**

AGENDA

A step-by-step path from pain to practice

01

The pain

Why technical documentation decays — and why that is a maintenance problem.

02

RAG

The first instinct: search the docs with an LLM. What it fixes, what it can’t.

03

LLM Wiki

Compile knowledge once, keep it current. Architecture and the ingest loop.

04

OKF

An open format so the wiki is portable, checkable and tool-independent.

05

knowledgeAgent

This project: a lifecycle that makes LLM-written knowledge trustworthy.

01

The pain of technical documentation

We are good at writing documents. We are bad at keeping them true.

THE PAIN

Six ways technical documentation lets us down

Scattered

Wikis, SharePoint, PDFs, tickets, code comments, chat threads. No single place to look — so people don't.

Stale

The code moved on; the doc didn't. Nothing tells you *when* it stopped being true.

Untrusted

No signal separating "verified last week" from "written years ago by someone who has left".

Contradictory

Three pages, three answers. Which one wins? Usually: whoever you asked last.

Tribal

The real knowledge lives in a few heads — and walks out of the door with them.

Write-only

Written for the release milestone, then never read, linked, or updated again.

THE PAIN

How a technical document dies — a story every team knows

DAY 1

Written & reviewed

The interface spec is accurate, complete, approved.



Matches reality

MONTH 2

Code changes

A sprint refactors the interface. Updating the spec isn't in the Definition of Done.

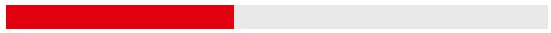


Silently drifting

MONTH 6

Someone trusts it

A new engineer follows the spec and builds against behaviour that no longer exists.

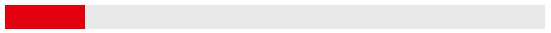


Actively misleading

MONTH 12

Nobody trusts it

The team stops reading the wiki and asks people instead. Knowledge becomes tribal again.



Abandoned

Docs don't fail at writing time. They fail at maintenance time — and nothing tells us when it happens.

The bottleneck is the bookkeeping, not the thinking

What people are good at

- Deciding which sources matter
- Judging whether a claim is right
- Asking the questions worth answering
- Seeing what it all means for the product

What people quietly stop doing

- Updating every page a change touches
- Noticing that a page went stale
- Reconciling two pages that disagree
- Maintaining cross-links and the index
- Writing down what we just learned

The tedious right-hand column is exactly the work an LLM doesn’t get tired of.

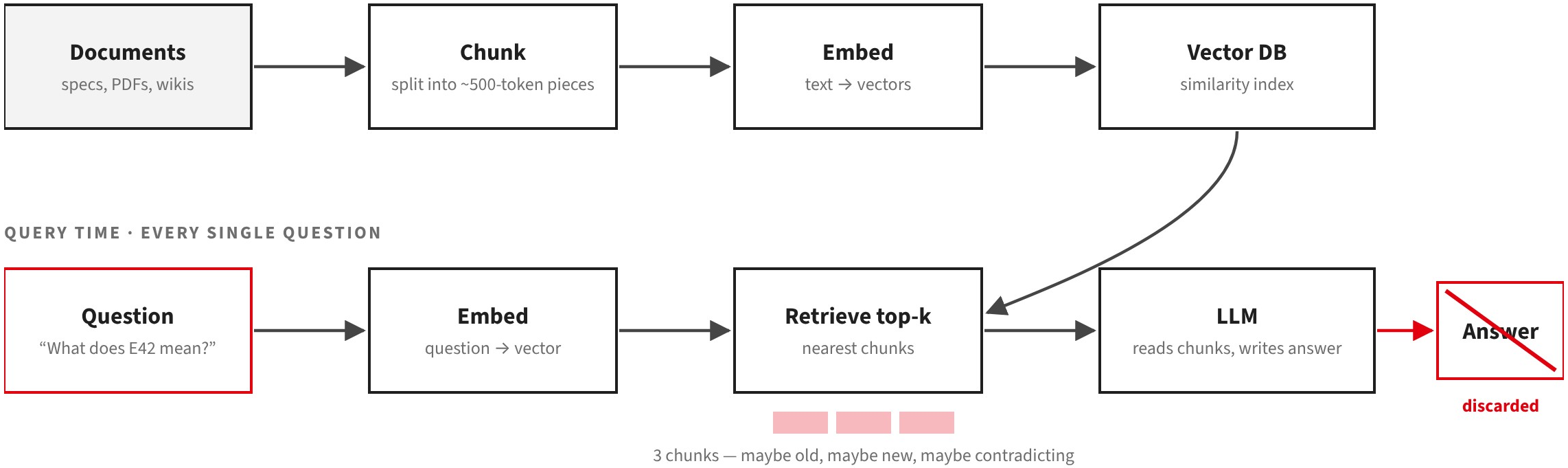
02

First instinct: put an LLM on top — RAG

Retrieval-Augmented Generation. Useful, popular — and not a cure for decay.

How RAG works

INDEX TIME · ONCE PER DOCUMENT



Every question starts from zero. The synthesis is thrown away; the 100th question is as hard as the 1st.

Where RAG falls short on engineering documentation

✕ Nothing accumulates

Knowledge is re-derived on every query. Insights from yesterday's answer are gone.

✕ Old and new coexist

Retrieval happily mixes the outdated spec with the current one. Similarity ≠ truth.

✕ Opaque

You can't open a vector index and review what the system "knows".

✕ Chunks lose context

A paragraph cut from its page — the caveat two sections up — is easy to misread.

✕ Contradictions stay invisible

Conflicting chunks are resolved (or not) silently inside one prompt; nobody is told.

✕ Garbage in, garbage out

It amplifies the documentation problem rather than fixing it.

RAG is a great search engine over your docs. It is not a maintainer of your docs.

03

The LLM Wiki pattern

Stop asking the LLM to re-read everything. Ask it to keep a wiki.

Compile knowledge once — don’t re-interpret it on every question

RAG ≈ INTERPRETER

Raw docs are re-read at query time

1. Question arrives
2. Fetch raw chunks
3. LLM synthesises an answer
4. Answer is shown… and forgotten

Cost and quality are paid again on every question.

LLM WIKI ≈ COMPILER



Knowledge is compiled at ingest time

1. A source arrives
2. LLM integrates it into wiki pages
3. Questions read the compiled pages
4. Good answers are filed back into the wiki

Knowledge compounds. Synthesis, cross-links and contradictions are already worked out before anyone asks.

Three layers, three operations

Schema
THE RULES

Conventions the agent follows: page types, frontmatter, naming, workflows. Co-evolved by people and the LLM.

`CLAUDE.md` `AGENTS.md` `SCHEMA.md`

Wiki
LLM-OWNED

Markdown pages the LLM writes and maintains: summaries, concepts, entities, comparisons — densely cross-linked.

`index.md` `log.md` `concepts/` `entities/` `comparisons/` `queries/`

Raw sources
IMMUTABLE

The evidence: specs, articles, tickets, transcripts. The LLM reads them — never edits them.

`sources/2026-11-08-spec.md` `sources/assets/`

Ingest WRITE

Source → summary, update related pages, index and log. One source can touch 10–15 pages.

Query READ

Search the wiki, answer with citations, file good answers back.

Lint AUDIT

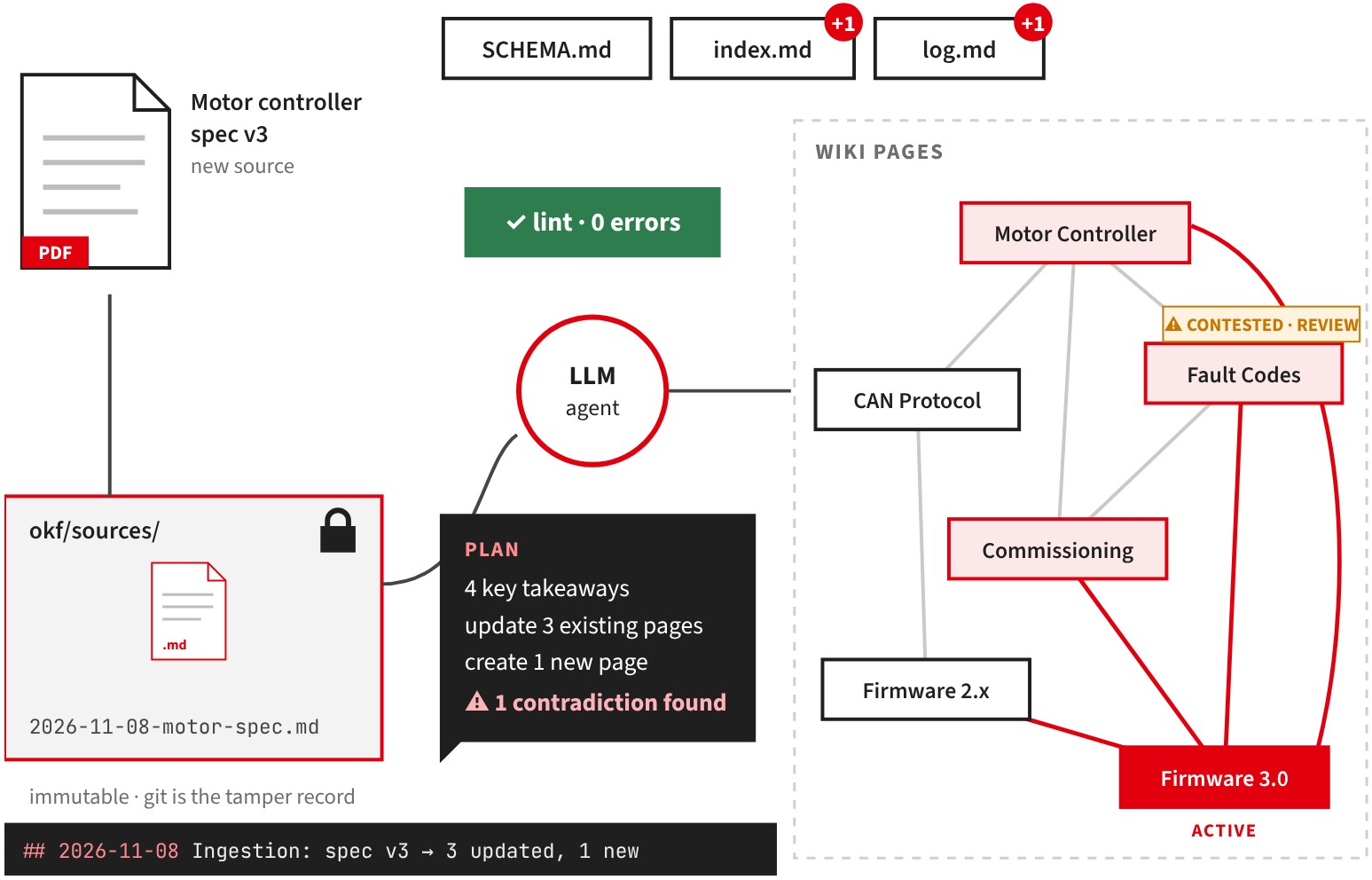
Contradictions, stale claims, orphans, missing links, gaps.

You

Curate sources, ask questions, review — and think about what it means.

Ingest, step by step

1. **Source arrives**
   a spec, ticket, article or conversation
2. **Snapshot it, immutably**
   `sources/YYYY-MM-DD-slug.md`
3. **Orient**
   read schema, index and recent log
4. **Discuss**
   takeaways, pages to touch, conflicts
5. **Write**
   update beats create; flag contradictions
6. **Cross-link**
   wire new knowledge into the graph
7. **Promote & record**
   draft → active, index + log
8. **Lint**
   mechanical health check must pass



One source touches many pages — that is where the compounding comes from

Answers that compound, and a wiki that checks itself

Query

1. Read `index.md` — the map of the wiki
2. Open only the relevant pages
3. Answer with citations to pages and sources
4. Flag anything under review or deprecated
5. File a substantial answer back as a page

```
E42 means over-temperature on the power stage [spec-v3].
⚠ /concepts/fault-codes.md is under review: spec v2
defined E42 differently; the conflict is flagged.
```

Lint

A periodic health check — partly mechanical, partly judgment.

Contradictions
pages that disagree

Stale claims
past their review date

Orphans
pages nothing links to

Missing links
concepts mentioned, never linked

Broken frontmatter
missing required fields

Gaps
questions worth a new source

Side by side

	RAG	LLM WIKI
When synthesis happens	At query time — every time	At ingest time — once, then maintained
Unit of knowledge	Text chunk, cut by size	A page per concept or entity, written for humans
Accumulation	None — answers are discarded	Compounds — sources and good answers enrich the wiki
Contradictions	Retrieved side by side, silently	Detected at ingest, flagged on the page
Freshness	Re-index; stale chunks linger	Pages are updated; stale ones are flagged
Auditability	Opaque embeddings	Readable, diffable, reviewable in a pull request
Infrastructure	Embedding model + vector database	Markdown files + git (search tool later, optional)
Cost profile	Cheap ingest, pay per query	Expensive ingest, cheap queries
Sweet spot	Huge, fast-moving, long-tail corpora	A curated core: hundreds to a few thousand pages

Honest trade-offs — and why they work better together

RAG

STRENGTHS

- Scales to millions of documents
- Near-zero curation effort, fast to stand up
- Good for long-tail “where was that written?” lookups

WEAKNESSES

- No memory, no synthesis that persists
- Inherits every flaw of the underlying docs

LLM Wiki

STRENGTHS

- Compounding, pre-synthesised knowledge
- Contradictions and gaps surface early
- Plain files: human-readable, git-reviewable

WEAKNESSES

- Ingest costs tokens and human attention
- **LLM mistakes persist** — the wiki needs a trust model

Use both: the wiki is the curated core; search or RAG covers the raw long tail — and the wiki itself, once it grows.

04

OKF — an open format for the wiki

The pattern is abstract on purpose. A team needs something concrete to agree on.

FROM PATTERN TO PRACTICE

The LLM Wiki pattern leaves two gaps for a team

GAP 1

No shared format

Every tool invents its own page layout and metadata. The wiki gets locked to one agent, one plugin, one person’s habits.

→ **OKF** standardises the files.

GAP 2

No trust model

An LLM-written page looks equally confident on day 1 and day 400. Write-time errors persist; drift goes unnoticed.

→ **The lifecycle profile** (this project) makes trust explicit.

OKF: markdown + YAML frontmatter, one hard rule

- A **bundle** is a directory of markdown files with YAML frontmatter.
- The only hard rule: every page has a **non-empty** `type`.
- Reserved files: `index.md` (catalog) and `log.md` (history, newest first).
- v0.2 standardises **provenance** (`sources`), **trust** (`generated`, `verified`) and **lifecycle** (`status`, `stale_after`).
- Readers must be **permissive**: unknown keys preserved, broken links tolerated — so profiles can extend it safely.
- Spec: GoogleCloudPlatform/open-knowledge-format

```
---
type: Entity
title: Motor Controller
description: Drive unit for …
status: stable
stale_after: 2027-02-08
sources:
  - resource: /sources/2026-11-08-spec.md
    id: spec-v3
generated: {by: claude-code, at: 2026-11-08}
lifecycle: active   # extension key
---
# Motor Controller
Rated for … [spec-v3]
```

OPEN KNOWLEDGE FORMAT

Why a plain, open format matters for engineering teams

Portable

Any agent or tool can read it — Claude Code, Copilot, Codex, a Python script, a human in an editor.

Git-native

Diff, blame, branch, review in pull requests. Git becomes the audit trail of what we know.

Human-readable

It's just markdown. No database, no vendor, no export step.

Machine-checkable

Frontmatter can be linted in CI, so structure is enforced before merge.

Extensible

Extra keys ride along; plain readers ignore them harmlessly.

Provenance built in

Every claim can point at its source, and say who produced or verified it.

05

knowledgeAgent — this project

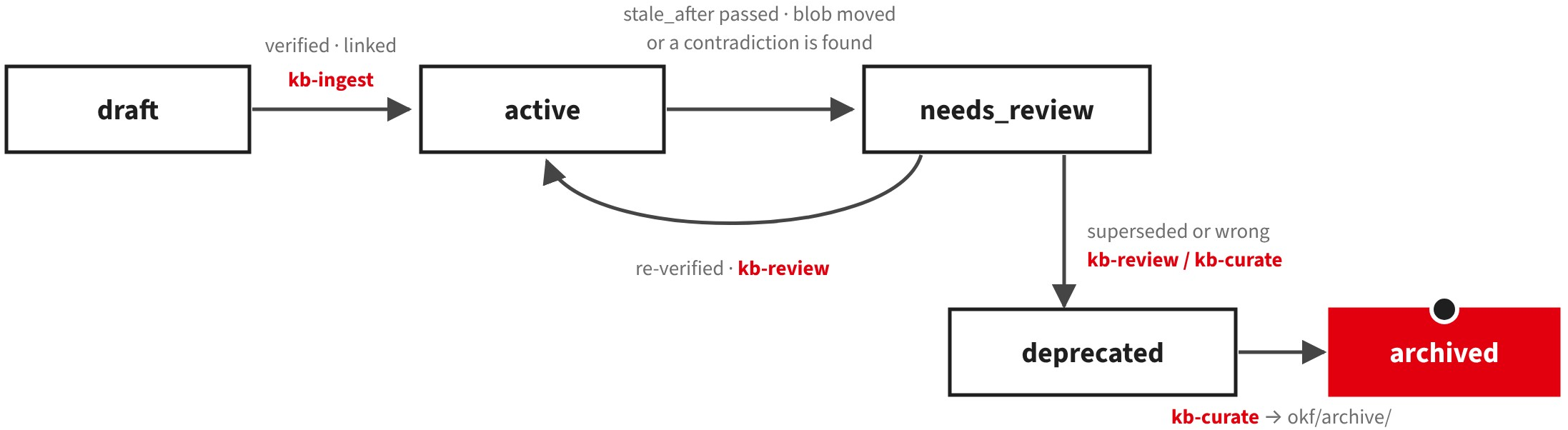
An OKF bundle, extended with a lifecycle profile, maintained by agent skills.

The whole project on one slide

Agents ANY OF THEM	Claude Code · GitHub Copilot · Codex · 70+ agents via `npx skills add`
Skills JUDGMENT	`kb-setup` · `kb-ingest` · `kb-query` · `kb-review` · `kb-curate` · `kb-lint`
okf.py MECHANICS · STDLIB	`init` · `lint` · `index` · `links` · `log` · `stats` · `drift`
okf/ THE BUNDLE	`SCHEMA.md` · `index.md` · `log.md` · `concepts/` · `entities/` · `comparisons/` · `queries/` · `archive/` · `sources/` 🔒

Design rule: judgment lives in the skills, mechanics live in the script. The script never decides whether a claim is *true*.

Knowledge is perishable: every page has a lifecycle



draft
captured, not yet verified or linked

active
trustworthy; must carry `stale_after`

needs_review
still served, but flagged in every answer

deprecated → archived
never deleted while anything links to it

Six upgrades on top of the LLM Wiki pattern

1

Lifecycle states

Five states with enforced exit criteria — OKF alone has no trust model.

2

Expiring truth

Active pages carry `stale_after`; overdue pages feed an automatic review queue.

3

Evidence pinning

A claim resting on a repo file pins its git `blob`; `okf.py drift` reopens the page when the file changes.

4

Immutable sources

Captures are append-only. A re-ingest is a new dated file; git history exposes any edit.

5

Graph-aware deprecation

`superseded_by` plus inbound-link checks — links are never silently broken.

6

Governed vocabulary

Page types and tags registered in `SCHEMA.md` before use; lint enforces it.

When the code changes, the page knows

`okf/entities/okf-toolchain.md` ● active

```yaml
---
type: Entity
title: okf.py — the Mechanical Toolchain
status: stable
lifecycle: active
sources:
  - resource: ../../scripts/okf.py
    id: okf-py
    blob: e81c07fa4b92   # pinned
verified:
  - {by: human:maintainer, at: …}
---
```

Real page from this repo. Values after step 1 are illustrative.

1. **Someone changes the file**
   A commit edits `scripts/okf.py` — the file this page's claims rest on.

2. **`okf.py drift` notices**
   It re-hashes every pinned file: `moved: scripts/okf.py`. Lint raises a warning — in CI, on the PR.

3. **The page is flagged**
   It goes to `needs_review`. It is still served, but every answer citing it carries a warning.

4. **`kb-review` re-verifies**
   Re-reads the evidence, fixes claims, re-pins the blob, appends `verified` → back to active.

One skill per lifecycle phase — plain English triggers

PHASE	SKILL	YOU SAY…	IT DOES…
Bootstrap	`kb-setup`	“Set up the knowledge base”	Scaffolds `okf/` and drops in `okf.py`
Capture	`kb-ingest`	“Ingest this spec”	Immutable snapshot → draft pages → cross-links → active → index + log
Serve	`kb-query`	“What do we know about E42?”	Answers with citations and trust labels; files good answers back
Verify	`kb-review`	“Work the review queue”	Re-checks overdue or drifted pages; resolves contradictions
Garden	`kb-curate`	“Clean up duplicates”	Merges, splits, deprecates, archives — keeps links whole
Audit	`kb-lint`	“Check the bundle”	Runs `okf.py lint` and fixes what it reports

Defined once in `skills/`; `okf.py init` generates the Claude Code copies and the Copilot prompt files from the same source.

Shared via git, guarded at merge

Tier 1 BLOCKING CI · SECONDS	`okf.py lint` · `sources/ is add-only` · `bundle change ⇒ log.md change` · `index.md is fresh`
Tier 2 ADVISORY LLM REVIEW	duplicate pages · unflagged contradictions · claims the cited source doesn't support
Tier 3 HUMAN GATES	CODEOWNERS on SCHEMA.md, skills/, okf.py · one teammate for ordinary pages

Block on cheap deterministic checks; keep judgment advisory. Conformance at merge, truth via the lifecycle.

KNOWLEDGEAGENT · STATUS

The bundle today — it maintains its own design notes

24

documents in the bundle

7 concepts · 6 entities · 2 comparisons · 8 sources · schema

13

pages active

2 needs_review · 1 deprecated, kept for its links

6

pinned repo files

0 drifted at the time of writing

11

pages in the review queue

past `stale_after` — the backstop is doing its job

NON-SOURCE PAGES BY LIFECYCLE



Topics so far: OKF itself, the lifecycle, PR quality gates, ingesting requirements from ticket systems, and comparisons with neighbouring tools (openwiki, codebase-memory-mcp).

Try it in your own project

1 · Install the skills

```
npx skills add yishao1224-design/knowledgeAgent \
  --skill kb-setup --skill kb-ingest --skill kb-query \
  --skill kb-review --skill kb-curate --skill kb-lint
```

2 · Gate it in CI

```
# exits 1 on errors
python scripts/okf.py lint
python scripts/okf.py drift
```

3 · Then just talk to your agent

- “Set up the knowledge base”
- “Ingest this design spec”
- “What do we know about fault E42?”
- “Work the review queue”

Start small.
One team, one domain, one bundle. Ingest what you already re-explain every week — and let it compound.

TAKEAWAYS

Five things to remember

1 **Docs fail at maintenance time, not writing time.**

The bottleneck is bookkeeping — exactly what LLMs are good at.

2 **RAG searches knowledge. An LLM Wiki maintains it.**

Compile once, keep current, let answers compound. Use both.

3 **OKF makes the wiki portable and checkable.**

Plain markdown + frontmatter, readable by any agent, reviewable in git.

4 **A lifecycle makes LLM-written knowledge trustworthy.**

Knowledge expires, evidence is pinned, sources are immutable.

5 **knowledgeAgent is ready to try.**

Six skills, one stdlib script, one bundle — in any repo, with any agent.

Q&A

Thank you

github.com/yishao1224-design/knowledgeAgent

gist.github.com/karpathy/442a6bf555914893e9891c11519de94f